

TypeScript Cheat Sheet

15 tables · 162 concepts · Updated 2026-07-16 · [cheatgrid.com](#)

TypeScript is a typed superset of JavaScript that adds a **static type system** and compile-time checks while still running as plain JavaScript after compilation. It matters because it lets teams model data shapes, APIs, and module boundaries explicitly, catching many bugs before runtime and improving editor tooling (navigation, refactors, autocomplete). The key mental model is that TypeScript's checking is mostly **structural** and happens at compile time — your types don't exist at runtime unless you write runtime validation. TypeScript 5.x has added significant capabilities including Stage 3 decorators, explicit resource management (`using` / `await using`), the `NoInfer<T>` utility type, inferred type predicates, and `--erasableSyntaxOnly` for Node.js type-stripping workflows.

CONTENTS

Type Annotations & Primitive Types	1
Object Shapes (Interfaces, Aliases, Properties)	2
Type Composition (Unions, Intersections, Literals, Enums)	3
Functions (Signatures, Generics, Overloads)	3
Classes (Members, Modifiers, Inheritance)	4
Narrowing (Type Guards & Control Flow)	5
Type Operators & Advanced Types	5
Utility Types (Built-in)	6
Decorators	7
Modules & Imports	8
JSX / TSX	9
TSCore (Type Checking, Emit, Projects)	10
tsc CLI Workflows	11
Declaration Files & Augmentation	11
Explicit Resource Management	12
References	13

TABLE 1

Type Annotations & Primitive Types

The starting point: how you attach a type to a value, and the built-in primitives you'll annotate with most often. Often you don't write the type at all, since inference reads it from the initializer, so the real skill is knowing when an explicit annotation earns its keep. Every type here is erased at compile time and none of it survives into the JavaScript you ship. Pay special attention to the three special types at the bottom: `any` switches checking off, `unknown` is the safe top type you must narrow first, and `never` marks code that can't produce a value.

Concept	Example	Description
Type annotation	<code>let n: number = 42</code>	- Explicitly assigns a static type to a variable, parameter, or return value - erased at compile time, so it never changes runtime behavior.
Type inference	<code>const s = "hello"</code>	Compiler infers a type from an initializer or usage without an annotation <code>const</code> keeps the literal type <code>"hello"</code> , <code>let</code> widens to <code>string</code> .
string	<code>let name: string = "Ada"</code>	- Primitive text type - Always the lowercase <code>string</code> , not the rarely used built-in <code>String</code> .
number	<code>let pi: number = 3.14159</code>	Primitive numeric type (a double-precision float). JS has no <code>int</code> or <code>float</code> , so <code>number</code> covers both.

Concept	Example	Description
boolean	<code>let ok: boolean = true</code>	- Primitive true/false type - An alias for the union <code>true false</code>.
bigint	<code>const id: bigint = 123n</code>	- Primitive for arbitrary-size integers (JS <code>bigint</code>) - cannot be mixed with <code>number</code> in arithmetic.
symbol	<code>const key: unique symbol = Symbol("k")</code>	- Primitive for unique identifiers <code>unique symbol</code> supports nominal-like keys, and is allowed only on <code>const</code> and <code>readonly static</code>.
null & undefined	<code>let x: string undefined</code>	- Represent absence - require explicit handling under <code>strictNullChecks</code>.
any	<code>let data: any = JSON.parse(text)</code>	Disables type checking for the value (escape hatch) - spreads to every property you read off it, so avoid in new code.
unknown	<code>let data: unknown = JSON.parse(text)</code>	- Safer top type than <code>any</code> - accepts any value, but must be narrowed before property access or calls.
void	<code>function log(msg: string): void { console.log(msg) }</code>	Indicates a function returns no useful value . Not the same type as <code>undefined</code> .
never	<code>function fail(msg: string): never { throw new Error(msg) }</code>	Indicates no possible value : unreachable code, always-throws, or an exhausted union.

TABLE 2

Object Shapes (Interfaces, Aliases, Properties)

Most real-world typing is describing the shape of objects, and these are the tools for it. Interfaces and type aliases both name a shape: interfaces can be re-opened and merged, while aliases can name any type expression at all, not just objects. On top of either you layer optional and readonly properties, index signatures for open-ended keys, and call or construct signatures for objects that are also callable. The excess property check is the quirk worth remembering: object literals get scrutinized for stray keys in ways stored variables don't.

Pattern	Example	Description
Interface	<code>interface User { id: string; name: string }</code>	- Named object shape - supports extension and declaration merging.
Type alias	<code>type User = { id: string; name: string }</code>	Gives a name to any type expression : primitives, unions, tuples, objects.
Object type literal	<code>const u: { id: string; name: string } = { id: "1", name: "Ada" }</code>	Inline property names and types without naming the type.
Optional property	<code>type User = { name: string; email?: string }</code>	Marks a property as possibly absent , adding <code>undefined</code> to its type.
Readonly property	<code>type User = { readonly id: string; name: string }</code>	Prevents assignment after initialization at the type-checking level.
Extending interfaces	<code>interface Admin extends User { role: "admin" }</code>	Builds larger shapes from smaller ones via <code>extends</code> .
Index signature	<code>type Headers = { [name: string]: string }</code>	Types objects whose keys aren't known in advance . Declared properties must match the value type.
Call signature	<code>type Fn = { (x: number): string }</code>	Describes a callable object that may also carry properties.
Construct signature	<code>type Ctor = { new (s: string): Date }</code>	Describes a new -able constructor type .
Excess property checks	<code>const u: User = { id: "1", name: "Ada", extra: 1 }</code>	Object literals checked for unknown properties when directly assigned to a type.

TABLE 3

Type Composition (Unions, Intersections, Literals, Enums)

This is where types start combining into something more expressive than a single shape. Unions say "one of these", intersections say "all of these at once", and literal types pin a value down to an exact string or number. Together they are the foundation of modeling states precisely. Tuples, `as const`, and the two flavors of enum round it out, with the notes flagging which constructs emit runtime code (string and numeric enums) versus which vanish at compile time.

Type	Example	Description
Union type	<code>type ID = string number</code>	Value can be one of multiple types.
Intersection type	<code>type WithId = { id: string } & { createdAt: Date }</code>	Value must satisfy all combined types simultaneously.
Const assertion (<code>as const</code>)	<code>const roles = ["admin", "user"] as const</code>	Infers the narrowest literal and readonly types for an expression.
String literal type	<code>type Role = "admin" "user"</code>	Restricts values to specific string constants .
Numeric literal type	<code>type Dice = 1 2 3 4 5 6</code>	Restricts values to specific numeric constants .
Tuple type	<code>const pair: [string, number] = ["a", 1]</code>	Fixed-length array with positional element types.
Non-null assertion (<code>!</code>)	<code>user!.id</code>	Asserts a value is not <code>null</code> / <code>undefined</code> for type checking (erased at runtime).
Enum (string)	<code>enum Role { Admin = "admin", User = "user" }</code>	- Members have explicit string values at runtime - readable when logged, but get no reverse mapping.
Enum (numeric)	<code>enum Status { Pending, Done }</code>	- Named constants that emit a runtime JS object - auto-increment from 0, with a reverse mapping back to names.
Const enum	<code>const enum Dir { Up, Down }</code>	Inlines enum values at emit time, leaving no runtime object - avoid in public declaration files.

TABLE 4

Functions (Signatures, Generics, Overloads)

Typing functions covers everything from a plain parameter list to generics that carry types through a call. Generics and their `extends` constraints are the heart of reusable, type-safe utilities, while overloads let one implementation present several distinct call signatures. The smaller entries (optional, default, and rest parameters, plus newer touches like `const` type parameters) handle the everyday details of getting arguments and return values typed cleanly.

Pattern	Example	Description
Function type expression	<code>type Mapper = (x: number) => string</code>	Describes a function's parameter and return types as a type alias - The parameter name is required
Generic function	<code>function first<T>(xs: T[]): T { return xs[0] }</code>	Parameterizes types with <code><T></code> for reusable , type-safe logic.
Generic constraint	<code>function len<T extends { length: number }>(x: T) { return x.length }</code>	Restricts a type parameter with <code>extends</code> so required members can be accessed.
Function overloads	<code>function parse(x: string): number;
function parse(x: number): string;
function parse(x: string number) { return typeof x === "string" ? +x : String(x) }</code>	Multiple call signatures with one implementation below them - the implementation signature is not callable from outside.
Optional parameter	<code>function greet(name?: string) {}</code>	Parameter may be omitted - typed as possibly <code>undefined</code> inside the function.

Pattern	Example	Description
Default parameter	<pre>function inc(n: number = 1) { return n + 1 }</pre>	Supplies a default value when the argument is missing or undefined , so the parameter is not <code>undefined</code> inside.
Rest parameter	<pre>function sum(...ns: number[]) { return ns.reduce((a,b)=>a+b, 0) }</pre>	Collects remaining arguments into a typed array . Must come last and be annotated as an array or tuple.
Const type parameter	<pre>function identity<const T>(val: T): T { return val } // T inferred as readonly ["a","b"], not string[]</pre>	Infers the narrowest literal type from a call-site argument (TS 5.0+). Only affects literals written inside the call.
Default type parameter	<pre>type ApiResult<T = unknown> = { data: T }</pre>	Provides a fallback when type arguments are not supplied or inferred.
this parameter	<pre>function f(this: Date) { return this.getTime() }</pre>	Fake first parameter to type <code>this</code> (erased in output, never passed as an argument).
Call signature + properties	<pre>type Fn = { (x: string): string; tag: string }</pre>	Models callable values that also carry properties . Uses <code>:</code> before the return type, not <code>=></code> .

TABLE 5

Classes (Members, Modifiers, Inheritance)

TypeScript classes are JavaScript classes with a type layer on top: the same fields, methods, and inheritance, plus visibility modifiers, `readonly`, and `implements` / `abstract` for enforcing contracts. Parameter properties save you a step by declaring and assigning a field straight from the constructor signature, and the `override` keyword (with `noImplicitOverride`) makes inheritance refactors safe. Watch the erasure boundary: `private` and `readonly` are compile-time checks that vanish from the emitted JavaScript, whereas JS `#` fields stay private at runtime.

Concept	Example	Description
Class declaration	<pre>class Point { x = 0; move(n: number) { this.x += n } }</pre>	• Defines fields/methods with type annotations • emits a JS class.
Parameter properties	<pre>class User { constructor(public readonly id: string) {} }</pre>	Shorthand that declares and initializes a member from a constructor parameter.
public / private / protected	<pre>class A { private secret = 1; protected n = 2; public id = "x" }</pre>	• Controls member visibility in type checking • use <code>#</code> for hard JS private fields.
readonly fields	<pre>class A { readonly id = "x" }</pre>	• Blocks assignment outside the constructor at type-check time • no runtime effect.
implements	<pre>class Repo implements Iterable<string> { [Symbol.iterator]() { return [][Symbol.iterator]() } }</pre>	Checks a class satisfies an interface at compile time.
extends (inheritance)	<pre>class B extends A {}</pre>	Subclassing with <code>super</code> calls and overriding.
abstract class	<pre>abstract class Base { abstract run(): void }</pre>	• Cannot be instantiated directly • may declare abstract members subclasses must implement.
Method override	<pre>class B extends A { override toString() { return "B" } }</pre>	• Replaces a base member • <code>override</code> keyword enforces the base member exists.
Accessors (get/set)	<pre>class A { get x() { return this._x } set x(v: number) { this._x = v } }</pre>	Declares typed property accessors .
Static initialization block	<pre>class C { static count = 0; static { C.count = 1 } }</pre>	Runs complex static setup code once during class initialization, with access to private members (TS 4.4+).
noImplicitOverride	<pre>// tsconfig.json
{ "compilerOptions": { "noImplicitOverride": true } }</pre>	Requires <code>override</code> keyword on all overriding members for safer refactors .

TABLE 6

Narrowing (Type Guards & Control Flow)

Narrowing is how a broad union type collapses to a specific one inside a branch. The compiler watches your control flow and tracks what a value can still be. The everyday guards reuse plain JavaScript checks (`typeof` , `instanceof` , `in` , truthiness, equality), and discriminated unions make this effortless by tagging each variant with a shared literal field. When the built-in checks aren't enough, type predicates and assertion functions let you teach the compiler your own narrowing logic, and the exhaustiveness pattern catches the case you forgot.

Technique	Example	Description
typeof narrowing	<pre>if (typeof x === "string") x.toUpperCase()</pre>	Narrows unions using JS <code>typeof</code> checks. Beware that <code>typeof null</code> is <code>"object"</code> .
instanceof narrowing	<pre>if (e instanceof Error) console.error(e.message)</pre>	Narrows via prototype chain checks. Needs a runtime value, so interfaces don't work.
in-operator narrowing	<pre>if ("id" in obj) obj.id</pre>	Narrows based on property existence. Optional properties stay in both branches.
Truthiness narrowing	<pre>if (s) s.trim()</pre>	• Narrows based on falsy/truthy JavaScript semantics • Also drops <code>""</code> , <code>0</code> , and <code>NaN</code> .
Equality narrowing	<pre>if (x === null) return</pre>	Narrows unions with <code>===</code> / <code>!==</code> comparisons. Loose <code>!= null</code> removes <code>null</code> and <code>undefined</code> at once.
Discriminated union	<pre>type Shape = { kind: "circle"; r: number } { kind: "square"; s: number }</pre>	• Uses a shared literal field (<code>kind</code>) to narrow via <code>switch</code> / <code>if</code> checks • A <code>string</code> field won't narrow
Type predicate	<pre>function isError(x: unknown): x is Error { return x instanceof Error }</pre>	• User-defined guard that narrows a value to a specific type • Unchecked, so the compiler trusts the claim
Inferred type predicate	<pre>const isString = (x: unknown) => typeof x === "string"</pre>	TS 5.5+ infers <code>x is string</code> when there's no explicit return type and a single <code>return</code> refines the parameter.
Assertion function (<code>asserts</code>)	<pre>function assertNonNull(x: unknown): asserts x { if (x == null) throw new Error() }</pre>	Narrows by asserting a condition holds when the function returns normally.
Exhaustiveness check	<pre>const _exhaustive: never = value</pre>	Forces the compiler to prove all union cases are handled. Errors at build time if a case is missed.

TABLE 7

Type Operators & Advanced Types

This is type-level programming: operators that compute new types from existing ones rather than just declaring them. `keyof` , `typeof` , and indexed access read structure out of other types; mapped types, conditional types, and `infer` transform and branch on it; template literal types build string types programmatically. It's the most powerful corner of TypeScript and the steepest, but it's what lets a library's types stay perfectly in sync with its data. The `satisfies` operator is a gentler standout, validating a value against a type without throwing away its narrower inferred shape.

Operator	Example	Description
Type assertion (<code>as</code>)	<pre>const input = document.getElementById("id") as HTMLInputElement</pre>	• Overrides TypeScript's inferred type • no runtime effect. Use when you know more than the compiler.
Double assertion	<pre>const x = value as unknown as TargetType</pre>	• Bypasses the "types don't sufficiently overlap" check via the <code>unknown</code> hop • still no runtime conversion. Use sparingly.
typeof (type context)	<pre>const cfg = { mode: "dev" as const }; type Cfg = typeof cfg</pre>	• Extracts a type from a runtime value's shape • Legal only on identifiers and their properties
keyof	<pre>type Keys = keyof User</pre>	• Produces a union of property keys from an object type • a string index signature yields <code>string number</code> , not literal keys.

Operator	Example	Description
Indexed access type	<code>type UserId = User["id"]</code>	Looks up a property type from another type via <code>T[K]</code> . The index is itself a type.
satisfies	<code>const routes = { home: "/" } satisfies Record<string, string></code>	Checks an expression matches a type without widening the expression's own inferred type.
Mapped type	<code>type RO<T> = { readonly [K in keyof T]: T[K] }</code>	• Creates a new object type by iterating over keys • mapping <code>keyof T</code> keeps <code>readonly</code> / <code>?</code> ; strip them with <code>-readonly</code> / <code>-?</code> .
Conditional type	<code>type IsString<T> = T extends string ? true : false</code>	Chooses a type based on an <code>extends</code> assignability test.
Template literal types	<code>type EventName = `on\${Capitalize<string>}`</code>	• Builds string types via template literal syntax over string unions • Unions in several slots cross multiply
Key remapping (as)	<code>type Getters<T> = { [K in keyof T as `get\${Capitalize<string & K>}`]: () => T[K] }</code>	• Renames or filters keys while mapping with the <code>as</code> clause • producing <code>never</code> drops the key.
Distributive conditional type	<code>type ToArray<T> = T extends any ? T[] : never</code>	• Distributes over unions when the checked type is a naked type parameter • wrap both sides (<code>[T] extends [U]</code>) to turn it off.
infer	<code>type Elem<T> = T extends (infer U)[] ? U : T</code>	• Introduces an inferred type variable inside a conditional type • In scope in the true branch only
Variance annotations (in / out)	<code>type ReadRef<out T> = { readonly value: T } type Writer<in T> = { write(x: T): void }</code>	• <code>out</code> = covariant (used in output position only) • <code>in</code> = contravariant (input only); checked, not a hint: a wrong annotation is an error. Improves checker accuracy and speed (TS 4.7+).
Recursive types	<code>type JSON = string number boolean null JSON[] { [k: string]: JSON }</code>	• A type alias that references itself • supported since TS 3.7 via deferred resolution at the alias's top level.

TABLE 8

Utility Types (Built-in)

These ship with TypeScript and save you from hand-writing the same transformations over and over. `Partial` , `Required` , `Readonly` , `Pick` , and `Omit` reshape object types; `Record` builds dictionaries; `Exclude` , `Extract` , and `NonNullable` filter unions; and `ReturnType` , `Parameters` , and `Awaited` extract pieces out of functions and promises. Two of them bite: `Partial` and `Readonly` only go one level deep, and `Omit` never checks your key against `keyof T` , so a typo silently removes nothing.

Type	Example	Description
Partial<T>	<code>type Patch = Partial<User></code>	Makes all properties of T optional (top level only).
Required<T>	<code>type StrictUser = Required<User></code>	Makes all properties of T required.
Readonly<T>	<code>type Frozen = Readonly<User></code>	Makes all properties of T readonly at compile time (top level only, nothing is frozen at runtime).
Pick<T, K>	<code>type UserRef = Pick<User, "id" "name"></code>	Selects a subset of properties by key.
Omit<T, K>	<code>type PublicUser = Omit<User, "email"></code>	• Removes properties by key • Keys aren't checked against <code>keyof T</code> , so a typo silently removes nothing
Record<K, T>	<code>type ById = Record<string, User></code>	Maps keys K to value type T .
Exclude<T, U>	<code>type NonString = Exclude<string number, string></code>	Removes from union T the members assignable to <code>U</code> .

Type	Example	Description
Extract<T, U>	<pre>type OnlyString = Extract<string number, string></pre>	Keeps from union T only the members assignable to U .
NonNullable<T>	<pre>type NN = NonNullable<string null undefined></pre>	Removes null and undefined from a type.
ReturnType<T>	<pre>type R = ReturnType<() => Promise<number>></pre>	Extracts a function type's return type.
Parameters<T>	<pre>type P = Parameters<(x: string, y: number) => void></pre>	Extracts a function type's parameter tuple.
Awaited<T>	<pre>type V = Awaited<Promise<string>></pre>	Recursively unwraps the resolved value type of a promise-like.
ConstructorParameters<T>	<pre>class C { constructor(a: string, b: number) {} } type P = ConstructorParameters<typeof C> // [string, number]</pre>	Extracts a constructor's parameter types as a tuple.
InstanceType<T>	<pre>type I = InstanceType<typeof User></pre>	Extracts the instance type of a constructor function type.
NoInfer<T>	<pre>function createStore<T>(init: T, fallback: NoInfer<T>): T { return init ?? fallback }</pre>	Blocks type inference from the wrapped position, forcing T to be inferred from other arguments (TS 5.4+).
ThisType<T>	<pre>const mixin: ThisType<{ x: number }> = { getX() { return this.x } }</pre>	Specifies the this type for methods in an object literal (requires <code>noImplicitThis</code>).
Uppercase<S> / Lowercase<S>	<pre>type U = Uppercase<"hello"> // "HELLO"</pre>	Intrinsic string types that transform string literal types to upper/lower case (TS 4.1+).
Capitalize<S> / Uncapitalize<S>	<pre>type C = Capitalize<"world"> // "World"</pre>	Intrinsic string types that capitalize or uncapitalize the first character of a string literal type (TS 4.1+).
ThisParameterType<T>	<pre>type T = ThisParameterType<(this: Date) => void> // Date</pre>	Extracts the this parameter type from a function type.
OmitThisParameter<T>	<pre>type F = OmitThisParameter<(this: Date, x: string) => void></pre>	Removes the this parameter from a function type.

TABLE 9

Decorators

Decorators wrap classes and their members to add or alter behavior, using the `@`-prefixed syntax you have seen in frameworks like Angular and NestJS. TypeScript 5.0 made the standardized TC39 decorators the default: each one receives the decorated value plus a context object and can return a replacement, and 5.2 added a metadata channel via `Symbol.metadata` . The crucial caveat is that these are not compatible with the older `experimentalDecorators` form, so a project picks one system for the whole compilation.

Decorator	Example	Description
Class decorator	<pre>function sealed(Base: Function, ctx: ClassDecoratorContext) { Object.seal(Base) } @sealed class C {}</pre>	• Wraps or replaces a class • receives the class and a context object • return a callable to replace the class, or nothing to keep it.
Method decorator	<pre>function log(fn: Function, ctx: ClassMethodDecoratorContext) { return function(this: any, ...args: any[]) { return fn.apply(this, args) } } class C { @log greet() {} }</pre>	• Wraps a method • the replacement function is returned • return nothing and the original method is left alone.

Decorator	Example	Description
Field decorator	<pre>function init<T>(val: T) { return (_, undefined, _ctx: ClassFieldDecoratorContext) => () => val } class C { @init("Ada") name!: string }</pre>	• Intercepts field initialization • receives <code>undefined</code>, never the field's value • return an initializer function to override the default value.
Accessor decorator	<pre>function bound({ get, set }: ClassAccessorDecoratorTarget<any, string>, ctx: ClassAccessorDecoratorContext) {} class C { @bound accessor name = "Ada" }</pre>	• Decorates an accessor keyword field, which auto-generates a getter/setter pair over a private slot • receives that <code>get</code> / <code>set</code> pair and may return <code>{ get, set, init }</code>.
Getter/setter decorator	<pre>function configurable(fn: Function, ctx: ClassGetterDecoratorContext) {} class C { @configurable get x() { return 1 } }</pre>	• Decorates a getter or setter separately • <code>@configurable</code> on <code>get x()</code> leaves <code>set x()</code> undecorated.
Decorator metadata	<pre>function meta(_: any, ctx: DecoratorContext) { ctx.metadata.info = "x" } @meta class C {} C[Symbol.metadata]</pre>	• TS 5.2+ adds <code>Symbol.metadata</code> to classes • decorators share one metadata object per class, accessible at runtime.
experimentalDecorators (legacy)	<pre>// tsconfig.json { "compilerOptions": { "experimentalDecorators": true } }</pre>	• Enables the pre-TC39 legacy decorator syntax used by Angular, NestJS, and older libraries • methods <code>get(target, propertyKey, descriptor)</code>, parameters can be decorated, and <code>emitDecoratorMetadata</code> works • incompatible with the standard decorators, and the flag applies to the whole compilation.

TABLE 10

Modules & Imports

How code is split across files and how TypeScript resolves and emits those boundaries. Standard ESM import/export is fully type-checked, and the type-only variants (`import type` / `export type`) mark symbols that exist purely for the type system and are erased from the output. The `tsconfig` cluster here (`moduleResolution`, `module`, `esModuleInterop`, `paths`) reconciles TypeScript with the realities of Node.js and bundlers, and is a frequent source of confusing import errors. Watch the emit boundary: `paths` and `allowSyntheticDefaultImports` change type checking only, so code that compiles cleanly can still fail at runtime.

Pattern	Example	Description
ES module import/export	<pre>import { readFile } from "node:fs/promises"</pre>	• Standard ECMAScript module syntax, fully type-checked by TS • Any file with a top-level <code>import</code> or <code>export</code> is a module • without one it is a script in the global scope
Type-only import (<code>import type</code>)	<pre>import type { User } from "./types"</pre>	Imports a symbol only for the type system , guaranteed to be erased from the emitted JavaScript.
Type-only export (<code>export type</code>)	<pre>export type { User } from "./types"</pre>	Re-exports types without a runtime export.
Import attributes	<pre>import data from "./data.json" with { type: "json" }</pre>	• Instructs the runtime how to load a module (e.g., JSON) • Contents are not checked by TS; they pass through to the host. Supersedes the deprecated <code>assert</code> syntax; supported from TS 5.3.
verbatimModuleSyntax	<pre>// tsconfig.json { "compilerOptions": { "verbatimModuleSyntax": true } }</pre>	Leaves any import/export without a <code>type</code> modifier in the output verbatim and drops anything marked <code>type</code> , so what you write is what you get .

Pattern	Example	Description
ESM/CJS interop	<code>import pkg from "cjs-only"</code>	- Covers how TS models default/namespace imports across ESM and CommonJS boundaries - Node gives every CJS module a synthetic default export of its whole <code>module.exports</code> .
esModuleInterop	<code>// tsconfig.json
 { "compilerOptions": { "esModuleInterop": true } }</code>	- Adjusts emit and checking for default/namespace imports to match Node.js/bundler behavior - Adds <code>__importDefault</code> / <code>__importStar</code> helpers and implies <code>allowSyntheticDefaultImports</code> .
allowSyntheticDefaultImports	<code>// tsconfig.json
 { "compilerOptions": { "allowSyntheticDefaultImports": true } }</code>	Allows <code>default</code> imports from modules without a default export (type-checking convenience only).
moduleResolution	<code>// tsconfig.json
 { "compilerOptions": { "moduleResolution": "bundler" } }</code>	- Selects how imports are resolved: <code>node16</code> , <code>nodenext</code> , or <code>bundler</code> for modern tooling. <code>bundler</code> never requires file extensions on relative imports - the Node modes do
module	<code>// tsconfig.json
 { "compilerOptions": { "module": "nodenext" } }</code>	Selects the emitted module format and influences resolution behavior.
baseUrl / paths	<code>// tsconfig.json
 { "compilerOptions": { "baseUrl": ".", "paths": { "@/*": ["src/*"] } } }</code>	Adds path mapping for module specifiers during type checking only . TS does not rewrite the specifier in emitted JS, so an alias needs a bundler or <code>tsc-alias</code> to work at runtime. <code>paths</code> values resolve relative to <code>baseUrl</code> when it is set.

TABLE 11

JSX / TSX

If you write React or any JSX-based UI, these are the knobs that make it type-check. The `.tsx` extension turns on JSX parsing, the `jsx` compiler option decides whether that syntax is transformed or preserved, and `jsxImportSource` points the automatic runtime at the right factory, so you can target Preact or another library instead of React. `JSX.IntrinsicElements` is where the set of valid built-in tags and their props lives.

Option	Example	Description
TSX file extension	<code>// Component.tsx
 export function Button() { return <button /> }</code>	- Enables JSX parsing in a source file - The <code>jsx</code> compiler option must be set too
jsx	<code>// tsconfig.json
 { "compilerOptions": { "jsx": "react-jsx" } }</code>	Controls how JSX is transformed or preserved by the compiler.
jsxImportSource	<code>// tsconfig.json
 { "compilerOptions": { "jsx": "react-jsx", "jsxImportSource": "preact" } }</code>	Sets the module for importing <code>jsx</code> / <code>jsx</code>s factories for the automatic runtime.
JSX.IntrinsicElements	<code>declare namespace JSX { interface IntrinsicElements { div: any } }</code>	Declares valid JSX tags and their prop types for type checking.

TABLE 12

TSConfig Core (Type Checking, Emit, Projects)

The compiler options you'll actually set in `tsconfig.json`, grouped around three jobs: how strictly to check, what to emit, and how to structure multi-package builds. `strict` is the single most consequential choice, but read it as a *named family* of flags rather than a master switch: `noUncheckedIndexedAccess` and `exactOptionalPropertyTypes` sit outside it and must be opted into by hand. The other trap worth internalising early is that `target` downlevels **syntax** and never adds polyfills, so a green build can still crash on an old engine.

Option	Example	Description
strict	<pre>// tsconfig.json
 { "compilerOptions": { "strict": true } }</pre>	Enables the whole strict family at once (<code>strictNullChecks</code> , <code>noImplicitAny</code> , <code>useUnknownInCatchVariables</code> , and others). The recommended baseline, but not every check: <code>noUncheckedIndexedAccess</code> and <code>exactOptionalPropertyTypes</code> stay opt-in.
strictNullChecks	<pre>// tsconfig.json
 { "compilerOptions": { "strictNullChecks": true } }</pre>	Treats <code>null</code> / <code>undefined</code> as distinct types requiring explicit handling.
noImplicitAny	<pre>// tsconfig.json
 { "compilerOptions": { "noImplicitAny": true } }</pre>	• Errors when the compiler would infer <code>any</code> due to missing type information • Explicit <code>any</code> is untouched
target	<pre>// tsconfig.json
 { "compilerOptions": { "target": "ES2022" } }</pre>	• Sets the emitted JS language level and downlevels syntax only. Never adds polyfills for runtime APIs • Also changes the default <code>lib</code>.
lib	<pre>// tsconfig.json
 { "compilerOptions": { "lib": ["ES2022", "DOM"] } }</pre>	Selects the built-in declaration libraries the checker loads (e.g., <code>DOM</code> , <code>ES2022</code>). Type definitions only: nothing from <code>lib</code> reaches your output.
skipLibCheck	<pre>// tsconfig.json
 { "compilerOptions": { "skipLibCheck": true } }</pre>	• Skips type checking of <code>.d.ts</code> files themselves, speeding up builds • their types still apply to your code.
resolveJsonModule	<pre>// tsconfig.json
 { "compilerOptions": { "resolveJsonModule": true } }</pre>	Allows importing <code>.json</code> files with full type inference from their content.
isolatedModules	<pre>// tsconfig.json
 { "compilerOptions": { "isolatedModules": true } }</pre>	Errors on patterns that require cross-file type information, making single-file transpile tools safe (e.g., esbuild, SWC). Changes no output.
declaration	<pre>// tsconfig.json
 { "compilerOptions": { "declaration": true } }</pre>	Emits a <code>.d.ts</code> per input file describing the public types of each module.
noEmit	<pre>// tsconfig.json
 { "compilerOptions": { "noEmit": true } }</pre>	• Runs the full type check without writing any output files • The standard CI check when another tool builds
incremental	<pre>// tsconfig.json
 { "compilerOptions": { "incremental": true } }</pre>	Saves project-graph info to a <code>.tsbuildinfo</code> file to speed up subsequent compilations.
composite	<pre>// tsconfig.json
 { "compilerOptions": { "composite": true } }</pre>	• Marks a project reference-able by other projects • Implies <code>declaration</code>, defaults <code>incremental</code> to true, and defaults <code>rootDir</code> to the config's folder
references	<pre>// tsconfig.json
 { "references": [{ "path": "../core" }] }</pre>	Declares dependencies between TS projects . Only <code>tsc -b</code> builds them: plain <code>tsc -p</code> reads their last-built <code>.d.ts</code> .
noUncheckedIndexedAccess	<pre>// tsconfig.json
 { "compilerOptions": { "noUncheckedIndexedAccess": true } }</pre>	Adds <code>undefined</code> to every indexed read (<code>arr[i]</code> , <code>obj[key]</code>). Deliberately not in <code>strict</code> , since it fires even in bounds-checked loops.

Option	Example	Description
exactOptionalPropertyTypes	<pre>// tsconfig.json
 { "compilerOptions": { "exactOptionalPropertyTypes": true } }</pre>	- Makes optional properties more exact: assigning <code>undefined</code> is rejected, distinct from the property being absent - Not part of <code>strict</code> .
useUnknownInCatchVariables	<pre>// tsconfig.json
 { "compilerOptions": { "useUnknownInCatchVariables": true } }</pre>	Types <code>catch (e)</code> as <code>unknown</code> instead of <code>any</code> , forcing you to narrow before use. Included in <code>strict</code> .
isolatedDeclarations	<pre>// tsconfig.json
 { "compilerOptions": { "isolatedDeclarations": true } }</pre>	- Requires exports to be annotated enough that other tools can generate <code>.d.ts</code> files without a type checker - Needs <code>declaration</code> or <code>composite</code> (TS 5.5+).
erasableSyntaxOnly	<pre>// tsconfig.json
 { "compilerOptions": { "erasableSyntaxOnly": true } }</pre>	Prohibits TS syntax with runtime semantics (enums, namespaces with runtime code, parameter properties, <code>import =</code>) so Node's type-stripping can just delete types (TS 5.8+).

TABLE 13

tsc CLI Workflows

The handful of `tsc` invocations that cover almost everything you'll do from the command line. `tsc --noEmit` is the one CI pipelines lean on, since it type-checks without writing a single output file, while `--watch` keeps recompiling as you edit and `tsc --init` scaffolds a fresh config. The `-b` build mode is the one to know once a codebase grows into multiple referenced projects, since it rebuilds them in dependency order. Watch the one big trap: naming any file on the command line makes `tsc` ignore `tsconfig.json` completely.

Command	Example	Description
Compile current project	<code>tsc</code>	Compiles using the nearest <code>tsconfig.json</code> , searching the current directory then up the parent chain - Naming files instead makes <code>tsc</code> ignore that config
Type-check only	<code>tsc --noEmit</code>	Runs the full type check while writing no JavaScript, declarations, or source maps.
Watch mode	<code>tsc --watch</code>	Recompiles on file changes continuously - It rebuilds only, it never re-runs your program
Initialize project	<code>tsc --init</code>	Creates a <code>tsconfig.json</code> scaffold with annotated options.
Compile by project file	<code>tsc -p tsconfig.build.json</code>	Compiles the project at an explicit config path, either a config file or a folder - Pass no input files alongside it
Build mode (project refs)	<code>tsc -b</code>	Builds a referenced project graph in dependency order, skipping projects already up to date.
Build with watch	<code>tsc -b --watch</code>	Incrementally rebuilds a project reference graph on changes.
Show TypeScript version	<code>tsc --version</code>	Prints the version of the compiler that is running.

TABLE 14

Declaration Files & Augmentation

Declaration files (`.d.ts`) carry types without any runtime code — they're how plain JavaScript packages describe their shapes and how you fill gaps in third-party types. Augmentation is the powerful part: `declare global` and module augmentation let you add members to existing types (a `userId` on Express's `Request` , say) without forking the original package, and declaration merging is the underlying mechanism that makes it work.

Pattern	Example	Description
Declaration file (.d.ts)	<pre>// index.d.ts
 export interface User { id: string }</pre>	Provides type information for JS at compile time with no runtime code.

Pattern	Example	Description
Global .d.ts	<pre>// globals.d.ts
 declare const VERSION: string</pre>	Declares globals accessible without imports (ambient global library style).
Module .d.ts	<pre>// index.d.ts
 declare module "pkg" { export function f(): void }</pre>	Declares the shape of an importable module for untyped JS packages.
declare global	<pre>declare global { interface Window { appVersion: string } }</pre>	Augments global types from within a module (requires at least one <code>import</code> or <code>export</code>).
Module augmentation	<pre>declare module "express-serve-static-core" { interface Request { userId?: string } }</pre>	Adds members to an existing module's declarations without forking the types package.
Declaration merging	<pre>interface Box { size: number } interface Box { color: string }</pre>	Multiple declarations with the same name are merged into one combined type.
Triple-slash reference (types)	<pre>/// <reference types="node" /></pre>	- Declares a type package dependency for the file - in <code>.ts</code> files, set <code>types</code> in <code>tsconfig.json</code> instead.
Triple-slash reference (lib)	<pre>/// <reference lib="dom" /></pre>	Explicitly includes a built-in lib (e.g., <code>dom</code>) in the compilation without changing <code>tsconfig.json</code> .

TABLE 15

Explicit Resource Management

Deterministic cleanup, built into the language: `try/finally` you no longer have to write by hand. Declare a resource with `using` (or `await using` for async cleanup) and its `[Symbol.dispose]` method runs automatically when the enclosing block exits, so connections and file handles close on an early `return` or a thrown error too. Resources are released in reverse order of declaration, the `Disposable` interface defines what makes a class eligible, and `DisposableStack` groups several resources to tear down together.

Feature	Example	Description
using declaration	<pre>using conn = getConnection()</pre>	- Calls <code>conn[Symbol.dispose]()</code> when the enclosing block exits, including on early <code>return</code> , <code>break</code> , and <code>throw</code> <code>null</code> / <code>undefined</code> are a no-op (TS 5.2+, ECMAScript Stage 4).
await using declaration	<pre>await using conn = await openConnection()</pre>	- Calls and awaits <code>conn[Symbol.asyncDispose]()</code> when the enclosing block exits - falls back to <code>[Symbol.dispose]</code> if there is no async disposer - the <code>await</code> marks disposal only, so acquisition needs its own <code>await</code> .
Symbol.dispose	<pre>class Conn { [Symbol.dispose]() { this.close() } }</pre>	- The synchronous cleanup method called by <code>using</code> - implement to make a class disposable.
Symbol.asyncDispose	<pre>class Conn { async [Symbol.asyncDispose]() { await this.close() } }</pre>	The asynchronous cleanup method called by <code>await using</code> .
Disposable interface	<pre>function use(r: Disposable) { using _ = r; }</pre>	Interface requiring <code>[Symbol.dispose]() : void</code> - available with <code>lib</code> set to <code>"esnext"</code> or <code>"esnext.disposable"</code> .
AsyncDisposable interface	<pre>async function use(r: AsyncDisposable) { await using _ = r; }</pre>	Interface requiring <code>[Symbol.asyncDispose]()</code> , returning a promise.
DisposableStack	<pre>using stack = new DisposableStack() stack.use(resource); stack.defer(() => cleanup())</pre>	Aggregates multiple resources for disposal in LIFO order <code>use()</code> takes a disposable, <code>defer()</code> a callback, <code>adopt()</code> a value plus its disposer <code>move()</code> transfers ownership <code>AsyncDisposableStack</code> for async variants.

Feature	Example	Description
SuppressedError	<code>err.error</code> / <code>err.suppressed</code>	• Wraps both errors when disposal throws while another error is already in flight • <code>error</code> is the later throw, <code>suppressed</code> the earlier one.

References

TypeScript official docs

1. **using** declaration - <https://www.typescriptlang.org/docs/handbook/release-notes/typescript-5-2.html#using-declarations-and-explicit-resource-management>
2. **abstract class** - <https://www.typescriptlang.org/docs/handbook/2/classes.html#abstract-classes-and-members>
3. **Accessors** (get/set) - <https://www.typescriptlang.org/docs/handbook/2/classes.html#getters--setters>
4. **allowSyntheticDefaultImports** - <https://www.typescriptlang.org/tsconfig/allowSyntheticDefaultImports.html>
5. **Assertion function** (`asserts`) - <https://www.typescriptlang.org/docs/handbook/release-notes/typescript-3-7.html#assertion-functions>
6. **Awaited**<T> - <https://www.typescriptlang.org/docs/handbook/utility-types.html#awaitedtype>
7. **baseUrl** / **paths** - <https://www.typescriptlang.org/docs/handbook/modules/reference.html>
8. **Call signature** - <https://www.typescriptlang.org/docs/handbook/2/functions.html#call-signatures>
9. **Class declaration** - <https://www.typescriptlang.org/docs/handbook/2/classes.html>
10. **Class decorator** - <https://www.typescriptlang.org/docs/handbook/release-notes/typescript-5-0.html>
11. **Compile by project file** - <https://www.typescriptlang.org/docs/handbook/tsconfig-json.html>
12. **Compile current project** - <https://www.typescriptlang.org/docs/handbook/compiler-options.html>
13. **composite** - <https://www.typescriptlang.org/tsconfig/composite.html>
14. **Conditional type** - <https://www.typescriptlang.org/docs/handbook/2/conditional-types.html>
15. **Const assertion** (`as const`) - <https://www.typescriptlang.org/docs/handbook/release-notes/typescript-3-4.html#const-assertions>
16. **Const enum** - <https://www.typescriptlang.org/docs/handbook/enums.html#const-enums>
17. **Const type parameter** - <https://www.typescriptlang.org/docs/handbook/release-notes/typescript-5-0.html#const-type-parameters>
18. **Construct signature** - <https://www.typescriptlang.org/docs/handbook/2/functions.html#construct-signatures>
19. **ConstructorParameters**<T> - <https://www.typescriptlang.org/docs/handbook/utility-types.html#constructorparameterstype>
20. **declaration** - <https://www.typescriptlang.org/tsconfig/declaration.html>
21. **Declaration file** (.d.ts) - <https://www.typescriptlang.org/docs/handbook/declaration-files/by-example.html>
22. **Declaration merging** - <https://www.typescriptlang.org/docs/handbook/declaration-merging.html>
23. **declare global** - <https://www.typescriptlang.org/docs/handbook/declaration-merging.html#global-augmentation>
24. **Decorator metadata** - <https://www.typescriptlang.org/docs/handbook/release-notes/typescript-5-2.html#decorator-metadata>
25. **Default type parameter** - <https://www.typescriptlang.org/docs/handbook/2/generics.html#generic-parameter-defaults>
26. **Discriminated union** - <https://www.typescriptlang.org/docs/handbook/2/narrowing.html#discriminated-unions>
27. **Disposable interface** - <https://www.typescriptlang.org/docs/handbook/release-notes/typescript-5-2.html#the-disposable-and-asyncdisposable-global-types>
28. **DisposableStack** - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DisposableStack
29. **Distributive conditional type** - <https://www.typescriptlang.org/docs/handbook/2/conditional-types.html#distributive-conditional-types>
30. **Enum (numeric)** - <https://www.typescriptlang.org/docs/handbook/enum.html#numeric-enums>
31. **Enum (string)** - <https://www.typescriptlang.org/docs/handbook/enums.html#string-enums>
32. **Equality narrowing** - <https://www.typescriptlang.org/docs/handbook/2/narrowing.html#equality-narrowing>
33. **erasableSyntaxOnly** - <https://www.typescriptlang.org/tsconfig/erasableSyntaxOnly.html>
34. **ES module import/export** - <https://www.typescriptlang.org/docs/handbook/2/modules.html>
35. **ESM/CJS interop** - <https://www.typescriptlang.org/docs/handbook/modules/appendices/esm-cjs-interop.html>
36. **esModuleInterop** - <https://www.typescriptlang.org/tsconfig/esModuleInterop.html>
37. **exactOptionalPropertyTypes** - <https://www.typescriptlang.org/tsconfig/exactOptionalPropertyTypes.html>
38. **Excess property checks** - <https://www.typescriptlang.org/docs/handbook/2/objects.html#excess-property-checks>
39. **Exclude**<T, U> - <https://www.typescriptlang.org/docs/handbook/utility-types.html#excludeuniontype-excludedmembers>
40. **Exhaustiveness check** - <https://www.typescriptlang.org/docs/handbook/2/narrowing.html#exhaustiveness-checking>
41. **experimentalDecorators** (legacy) - <https://www.typescriptlang.org/tsconfig/experimentalDecorators.html>
42. **Extending interfaces** - <https://www.typescriptlang.org/docs/handbook/2/objects.html#extending-types>
43. **extends** (inheritance) - <https://www.typescriptlang.org/docs/handbook/2/classes.html#extends-clauses>
44. **Extract**<T, U> - <https://www.typescriptlang.org/docs/handbook/utility-types.html#extracttype-union>
45. **Field decorator** - <https://github.com/tc39/proposal-decorators>
46. **Function overloads** - <https://www.typescriptlang.org/docs/handbook/2/functions.html#function-overloads>
47. **Function type expression** - <https://www.typescriptlang.org/docs/handbook/2/functions.html#function-type-expressions>
48. **Generic constraint** - <https://www.typescriptlang.org/docs/handbook/2/generics.html#generic-constraints>
49. **Generic function** - <https://www.typescriptlang.org/docs/handbook/2/generics.html>
50. **Global .d.ts** - <https://www.typescriptlang.org/docs/handbook/declaration-files/templates/global-d-ts.html>
51. **implements** - <https://www.typescriptlang.org/docs/handbook/2/classes.html#implements-clauses>
52. **Import attributes** - <https://www.typescriptlang.org/docs/handbook/release-notes/typescript-5-3.html>

- 53. in-operator narrowing - <https://www.typescriptlang.org/docs/handbook/2/narrowing.html#the-in-operator-narrowing>
- 55. Index signature - <https://www.typescriptlang.org/docs/handbook/2/objects.html#index-signatures>
- 57. infer - <https://www.typescriptlang.org/docs/handbook/2/conditional-types.html#inferring-within-conditional-types>
- 59. instanceof narrowing - <https://www.typescriptlang.org/docs/handbook/2/narrowing.html#instanceof-narrowing>
- 61. Interface - <https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#interfaces>
- 63. isolatedDeclarations - <https://www.typescriptlang.org/tsconfig/isolatedDeclarations.html>
- 65. jsx - <https://www.typescriptlang.org/tsconfig/jsx.html>
- 67. jsxImportSource - <https://www.typescriptlang.org/tsconfig/jsxImportSource.html>
- 69. keyof - <https://www.typescriptlang.org/docs/handbook/2/keyof-types.html>
- 71. Mapped type - <https://www.typescriptlang.org/docs/handbook/2/mapped-types.html>
- 73. module - <https://www.typescriptlang.org/tsconfig/module.html>
- 75. Module augmentation - <https://www.typescriptlang.org/docs/handbook/declaration-merging.html#module-augmentation>
- 77. never - <https://www.typescriptlang.org/docs/handbook/2/narrowing.html>
- 79. noImplicitAny - <https://www.typescriptlang.org/tsconfig/noImplicitAny.html>
- 81. NoInfer<T> - <https://www.typescriptlang.org/docs/handbook/release-notes/typescript-5-4.html#the-noinfer-utility-type>
- 83. NonNullable<T> - <https://www.typescriptlang.org/docs/handbook/utility-types.html#nonnullabletype>
- 85. Object type literal - <https://www.typescriptlang.org/docs/handbook/2/objects.html>
- 87. OmitThisParameter<T> - <https://www.typescriptlang.org/docs/handbook/utility-types.html#omitthisparametertype>
- 89. Optional property - <https://www.typescriptlang.org/docs/handbook/2/objects.html#optional-properties>
- 91. Parameters<T> - <https://www.typescriptlang.org/docs/handbook/utility-types.html#parameterstype>
- 93. Pick<T, K> - <https://www.typescriptlang.org/docs/handbook/utility-types.html#picktype-keys>
- 95. readonly fields - <https://www.typescriptlang.org/docs/handbook/2/classes.html#readonly>
- 97. Readonly<T> - <https://www.typescriptlang.org/docs/handbook/utility-types.html#readonlytype>
- 99. Recursive types - <https://www.typescriptlang.org/docs/handbook/release-notes/typescript-3-7.html>
- 101. Required<T> - <https://www.typescriptlang.org/docs/handbook/utility-types.html#requiredtype>
- 103. Rest parameter - <https://www.typescriptlang.org/docs/handbook/2/functions.html#rest-parameters-and-arguments>
- 105. satisfies - <https://www.typescriptlang.org/docs/handbook/release-notes/typescript-4-9.html#the-satisfies-operator>
- 107. Static initialization block - <https://www.typescriptlang.org/docs/handbook/2/classes.html#static-blocks>
- 109. strictNullChecks - <https://www.typescriptlang.org/tsconfig/strictNullChecks.html>
- 111. symbol - <https://www.typescriptlang.org/docs/handbook/symbols.html>
- 113. Symbol.dispose - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/dispose
- 115. Template literal types - <https://www.typescriptlang.org/docs/handbook/2/template-literal-types.html>
- 117. ThisParameterType<T> - <https://www.typescriptlang.org/docs/handbook/utility-types.html#thisparametertypetype>
- 119. Triple-slash reference (lib) - <https://www.typescriptlang.org/docs/handbook/triple-slash-directives.html#reference-lib>
- 54. incremental - <https://www.typescriptlang.org/tsconfig/incremental.html>
- 56. Indexed access type - <https://www.typescriptlang.org/docs/handbook/2/indexed-access-types.html>
- 58. Inferred type predicate - <https://www.typescriptlang.org/docs/handbook/release-notes/typescript-5-5.html#inferred-type-predicates>
- 60. InstanceType<T> - <https://www.typescriptlang.org/docs/handbook/utility-types.html#instancetypetype>
- 62. Intersection type - <https://www.typescriptlang.org/docs/handbook/2/objects.html#intersection-types>
- 64. isolatedModules - <https://www.typescriptlang.org/tsconfig/isolatedModules.html>
- 66. JSX.IntrinsicElements - <https://www.typescriptlang.org/docs/handbook/jsx.html#type-checking>
- 68. Key remapping (as) - <https://www.typescriptlang.org/docs/handbook/2/mapped-types.html#key-remapping-via-as>
- 70. lib - <https://www.typescriptlang.org/tsconfig/lib.html>
- 72. Method override - <https://www.typescriptlang.org/docs/handbook/2/classes.html#overriding-methods>
- 74. Module.d.ts - <https://www.typescriptlang.org/docs/handbook/declaration-files/templates/module-d-ts.html>
- 76. moduleResolution - <https://www.typescriptlang.org/tsconfig/moduleResolution.html>
- 78. noEmit - <https://www.typescriptlang.org/tsconfig/noEmit.html>
- 80. noImplicitOverride - <https://www.typescriptlang.org/tsconfig/noImplicitOverride.html>
- 82. Non-null assertion (!) - <https://www.typescriptlang.org/docs/handbook/release-notes/typescript-2-0.html#non-null-assertion-operator>
- 84. noUncheckedIndexedAccess - <https://www.typescriptlang.org/tsconfig/noUncheckedIndexedAccess.html>
- 86. Omit<T, K> - <https://www.typescriptlang.org/docs/handbook/utility-types.html#omittype-keys>
- 88. Optional parameter - <https://www.typescriptlang.org/docs/handbook/2/functions.html#optional-parameters>
- 90. Parameter properties - <https://www.typescriptlang.org/docs/handbook/2/classes.html#parameter-properties>
- 92. Partial<T> - <https://www.typescriptlang.org/docs/handbook/utility-types.html#partialtype>
- 94. public / private / protected - <https://www.typescriptlang.org/docs/handbook/2/classes.html#member-visibility>
- 96. Readonly property - <https://www.typescriptlang.org/docs/handbook/2/objects.html#readonly-properties>
- 98. Record<K, T> - <https://www.typescriptlang.org/docs/handbook/utility-types.html#recordkeys-type>
- 100. references - <https://www.typescriptlang.org/docs/handbook/project-references.html>
- 102. resolveJsonModule - <https://www.typescriptlang.org/tsconfig/resolveJsonModule.html>
- 104. ReturnType<T> - <https://www.typescriptlang.org/docs/handbook/utility-types.html#returntypetype>
- 106. skipLibCheck - <https://www.typescriptlang.org/tsconfig/skipLibCheck.html>
- 108. strict - <https://www.typescriptlang.org/tsconfig/strict.html>
- 110. String literal type - <https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#literal-types>
- 112. Symbol.asyncDispose - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/asyncDispose
- 114. target - <https://www.typescriptlang.org/tsconfig/target.html>
- 116. this parameter - <https://www.typescriptlang.org/docs/handbook/2/functions.html#declaring-this-in-a-function>
- 118. ThisType<T> - <https://www.typescriptlang.org/docs/handbook/utility-types.html#thistypetype>
- 120. Triple-slash reference (types) - <https://www.typescriptlang.org/docs/handbook/triple-slash-directives.html>

- 121.** Truthiness narrowing - <https://www.typescriptlang.org/docs/handbook/2/narrowing.html#truthiness-narrowing>
- 123.** Tuple type - <https://www.typescriptlang.org/docs/handbook/2/objects.html#tuple-types>
- 125.** Type annotation - <https://www.typescriptlang.org/docs/handbook/2/everyday-types.html>
- 127.** Type predicate - <https://www.typescriptlang.org/docs/handbook/2/narrowing.html#using-type-predicates>
- 129.** Type-only import (`import type`) - <https://www.typescriptlang.org/docs/handbook/modules/reference.html#import-type>
- 131.** typeof narrowing - <https://www.typescriptlang.org/docs/handbook/2/narrowing.html#typeof-type-guards>
- 133.** unknown - <https://www.typescriptlang.org/docs/handbook/2/functions.html>
- 135.** useUnknownInCatchVariables - <https://www.typescriptlang.org/tsconfig/useUnknownInCatchVariables.html>
- 137.** verbatimModuleSyntax - <https://www.typescriptlang.org/tsconfig/verbatimModuleSyntax.html>
- 122.** TSX file extension - <https://www.typescriptlang.org/docs/handbook/jsx.html>
- 124.** Type alias - <https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#type-aliases>
- 126.** Type assertion (`as`) - <https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#type-assertions>
- 128.** Type-only export (`export type`) - <https://www.typescriptlang.org/docs/handbook/modules/reference.html#export-type>
- 130.** typeof (type context) - <https://www.typescriptlang.org/docs/handbook/2/typeof-types.html>
- 132.** Union type - <https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#union-types>
- 134.** Uppercase<S> / Lowercase<S> - <https://www.typescriptlang.org/docs/handbook/2/template-literal-types.html#intrinsic-string-manipulation-types>
- 136.** Variance annotations (`in` / `out`) - <https://www.typescriptlang.org/docs/handbook/release-notes/typescript-4-7.html#optional-variance-annotations-for-type-parameters>
- 138.** Watch mode - <https://www.typescriptlang.org/docs/handbook/configuring-watch.html>